

Raport de Laborator: Gestionarea Persistenței și Migrării Datelor

1. Strategia de Migrare și Motivarea Alegerii

Pentru acest proiect, am implementat o **Strategie de Migrare Automatizată, Bazată pe Stări**, utilizând **Liquibase**.

Argumente:

- **Controlul Versiunilor:** Prin utilizarea fișierelor de tip *changelog* (XML), schema bazei de date este tratată ca și cod, permițând colaborarea eficientă și urmărirea istoricului modificărilor.
- **Independența de Baza de Date:** Liquibase abstractizează limbajul SQL; același *changelog* poate fi aplicat pe H2 (pentru dezvoltare locală) sau pe PostgreSQL/MySQL (pentru producție) fără a rescrie scripturile.
- **Siguranță:** Tabelul DATABASECHANGELOG previne executarea duplicată a aceluiași migrări și oferă un jurnal de audit clar pentru fiecare modificare aplicată.

2. Documentarea Execuției Migrării

Migrarea a fost executată într-o manieră modulară, fiind împărțită pe sarcini specifice:

- **Schema Inițială:** Crearea tabelelor de bază employee (angajat) și department.
- **Evoluția Schemei:** Adăugarea coloanelor pentru telefon, crearea tabelului projects cu o relație de tip *Many-to-Many* și modificarea preciziei coloanei salary.
- **Concurență și Logică:** Adăugarea coloanei version pentru blocare (locking) și a indicatorilor deleted pentru ștergerea logică (soft delete).

Pași de Execuție:

1. Configurarea build.gradle.kts cu runtime-ul Liquibase.
2. Rularea comenzii ./gradlew dbUpdate pentru aplicarea modificărilor.
3. Verificarea tabelului DATABASECHANGELOG pentru confirmarea execuției cu succes.

- **Cancel (Anulare):** Abandonarea completă a tranzacției.

4. Soft Delete vs. Hard Delete (Ștergere Logică vs. Fizică)

Caracteristică	Soft Delete (Ștergere Logică)	Hard Delete (Ștergere Fizică)
Retenția Datelor	Datele rămân în tabel cu un marcaj/flag.	Datele sunt eliminate permanent.
Recuperare	Ușor de restaurat (schimbarea unui boolean).	Necesită backup-uri ale bazei de date.
Audit	Menține un istoric (cine/când a șters).	Istoricul este pierdut.
Performanță	Tabelele cresc în timp; interogările necesită filtre.	Menține tabelele compacte și rapide.

Compromisuri: Am ales **Soft Delete** pentru entitatea Employee pentru a asigura integritatea datelor și posibilitatea de auditare. Folosind `@SQLDelete` și `@SQLRestriction` în Hibernate, ne-am asigurat că înregistrările șterse sunt ascunse utilizatorilor obișnuiți, rămânând accesibile administratorilor prin interogări native.

```
Database is up to date, no changes to execute
2024-05-13T19:26:30.552+03:00 main @DatabaseConfiguration @DatabaseConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Explicitly configure spring.jpa.open-in-view to disable this warning
Hibernate: select e1.id,e1.deleted,e1.deleted_at,e1.deleted_by,e1.id,e1.name,e1.email,e1.name,e1.phone,e1.salary,e1.version from employee e1,e2 left join department d1,e2 on e1.id=e1.d.department_id where e1.id=? and (e1.deleted = false)

*** STARTING SOFT DELETE DEMO ***
Deleting Employee ID 1...
Hibernate: select e1.id,e1.deleted,e1.deleted_at,e1.deleted_by,e1.id,e1.name,e1.email,e1.name,e1.phone,e1.salary,e1.version from employee e1,e2 left join department d1,e2 on e1.id=e1.d.department_id where e1.id=? and (e1.deleted = false)
Hibernate: UPDATE employee SET deleted = true, deleted_at = NOW(), deleted_by = 'admin_user' WHERE id = ? AND version = ?
Hibernate: select e1.id,e1.deleted,e1.deleted_at,e1.deleted_by,e1.id,e1.name,e1.email,e1.name,e1.phone,e1.salary,e1.version from employee e1,e2 left join department d1,e2 on e1.id=e1.d.department_id where e1.id=? and (e1.deleted = false)
Search for ID 1 (standard query) returned: NULL (Success)
Hibernate: SELECT * FROM employee WHERE Deleted = true
Admin View - Deleted Records count: 1
Audit Trail -> Deleted at: 2024-05-13T19:26:30.683059 by: admin_user
Restarting Employee ID 1...
Hibernate: SELECT * FROM employee WHERE id = ?
Employee 1 restored successfully.
Hibernate: update employee set deleted=?,deleted_at=?,deleted_by=?,department_id=?,email=?,name=?,phone=?,salary=?,version=? where id=? and version=?
Hibernate: select e1.id,e1.deleted,e1.deleted_at,e1.deleted_by,e1.id,e1.name,e1.email,e1.name,e1.phone,e1.salary,e1.version from employee e1,e2 left join department d1,e2 on e1.id=e1.d.department_id where e1.id=? and (e1.deleted = false)
Search for ID 1 after restoration: updated by user A
Process finished with exit code 0
```

5. Lecții Învățate și Provocări

Provocări Întâmpinate:

- **Contextul Tranzacțional:** O provocare majoră a fost starea "Rollback-only". Când apare o eroare de bază de date, Spring marchează tranzacția ca fiind compromisă. Am învățat să folosesc `Propagation.REQUIRES_NEW` pentru a efectua acțiuni de recuperare (precum reîncărcarea datelor) într-o tranzacție nouă.
- **Căile Liquibase (Pathing):** Am întâmpinat probleme cu *changelog*-uri duplicate în folderele `build/` și `src/`. Aceasta m-a învățat importanța gestionării *classpath*-ului în Gradle și utilizarea `duplicateFileMode=WARN`.

Bune Practici:

- Utilizarea întotdeauna a `entityManager.flush()` atunci când se testează concurența, pentru a forța verificarea bazei de date înainte de finalizarea tranzacției.
- Utilizarea interogărilor native pentru funcțiile de Administrare/Restaurare pentru a ocoli filtrele globale implicite.

6. Diagrama Schemei Bazei de Date

